Mini-moteur de recherche d'images basé sur le contenu

Séance 2

I. Introduction

Après avoir construit et harmonisé la base d'images lors de la première séance, nous allons approfondir la compréhension du contenu des images numériques.
L'objectif est de manipuler les images à un niveau plus bas, celui de leurs représentations numériques, afin de mieux appréhender les transformations possibles avant l'extraction de caractéristiques. Nous découvrirons les différents espaces de couleurs, apprendrons à convertir les images d'un espace à un autre, à visualiser leurs canaux, et à quantifier leurs couleurs.
Ces manipulations constituent une étape essentielle pour comprendre la manière dont les informations visuelles sont codées, transformées et exploitées dans un moteur de recherche d'images.

II. Plan de la séance

a. Représentation et exploration numérique d'une image

Lors de la séance précédente, nous avons appris à charger une image à partir d'un chemin (charger_image(chemin)), à l'afficher et à consulter quelques propriétés (dimensions, taille, ratio, …). Avant de pouvoir la traiter, il est essentiel de comprendre comment une image est représentée numériquement en mémoire et stockée sur disque. Une image numérique est un tableau de nombres (souvent un tableau NumPy sous Python) dont les dimensions sont de la forme *(hauteur, largeur, nombre_de_canaux)*. Chaque pixel est défini par un ou plusieurs canaux (Rouge, Vert, Bleu, …), et chaque canal contient une intensité lumineuse comprise généralement entre 0 et 255 pour les images codées sur 8 bits (uint8). Pour certaines images scientifiques, on peut avoir des entiers 16 bits ou des flottants (float32) pour plus de précision. Sur disque, l'image n'est pas stockée sous forme de tableau NumPy mais dans un format de fichier (PNG, JPEG, TIFF, …). Le fichier contient les données des pixels, éventuellement compressées ou non et des métadonnées (dimensions, type de pixel, profil de couleur, informations EXIF pour les photos…).

a. Chargement et inspection de base :

Lors de la première séance, vous avez déjà implémenté une fonction charger_image() (en Pillow ou en OpenCV) pour lire une image depuis le disque et la représenter sous forme de matrice de pixels.
Dans cette séance, nous allons approfondir la compréhension de la représentation des images en mémoire en comparant deux pproches de chargement **:** Pillow et OpenCV.

i. **Comparaison des bibliothèques Pillow et OpenCV :** Écrire deux fonctions :

- **charger_image_pillow(chemin)** : qui charge une image avec Pillow,

```
from PIL import Image

import numpy as np

def charger_image_pillow(chemin):

    img = Image.open(chemin)

    return np.array(img)
```

- **charger_image_cv2(chemin)** : qui charge une image avec OpenCV.

```
import cv2

def charger_image_cv2(chemin):

    img = cv2.imread(chemin)

    return img
```

- Afficher les deux images (côte à côte ou séparément) avec matplotlib.pyplot.imshow().
  - Qu'observez-vous ?
  - Qu'est-ce qu'il faut faire pour que les deux affichages deviennent identiques.

ii. **Inspection des propriétés d'une image :**

- Écrire une fonction **informations_image(image)** qui affiche :
  - les dimensions (hauteur, largeur, nombre de canaux),
  - le type de données (uint8, float32, etc.),
  - la valeur minimale et maximale des pixels.

iii. **Vérification de la nature et du type de l'image :**

- Écrire une fonction **type_image(image)** qui identifie la nature de l'image selon sa structure interne :
  - **Image binaire** : une seule couche, valeurs dans {0, 1}
  - **Image en niveaux de gris** : une seule couche, valeurs continues (0–255)
  - **Image couleur** : trois canaux (R, G, B ou B, G, R selon la bibliothèque)
  - **Image avec transparence** : quatre canaux (R, G, B, A)

iv. Dynamique et type :

- Tester les fonctions informations_image (image) et type_image(image) sur quelques images de la base harmonisée ou autres images sur votre disque ou téléchargées sur le web.

b. Introduction aux espaces de couleurs :

Au-delà du format de fichier « .png, .gif, .jpeg », une image peut être représentée selon différents espaces couleurs, qui définissent la manière dont les valeurs numériques traduisent les couleurs perçues. L'espace le plus courant est le RGB (RVB : Rouge, Vert, Bleu), utilisé pour l'affichage sur les écrans. Cependant, d'autres espaces comme HSV**,** CMYK ou Lab sont souvent employés selon le type d'application, qu'il s'agisse de traitement d'image, d'impression ou d'analyse visuelle. Comprendre ces espaces et savoir convertir une image de l'un à l'autre est essentiel pour maîtriser les effets visuels et les opérations de transformation dans le traitement et l'analyse et la description du contenu, d'images numériques.

i. Comprendre la représentation des couleurs :

Les espaces de couleurs sont des systèmes normalisés. Plusieurs organismes et standards existent parce que chaque technologie (écran, imprimante, scanners, appareil photo, smartphone, etc) ou domaine a ses besoins spécifiques (Impression, vidéo, design graphique, photographie, etc). Les principaux organismes et entreprises qui définissent les espaces de couleurs sont : CIE (Commission Internationale de l'Eclairage), ISO (Organisation internationale de normalisation), IEC (Commission électronique internationale), Apple, Adobe, Microsoft, etc. Chaque espace est adapté à des traitements particuliers.

RGB : représentation additive, sensible à la lumière.

HSV/HSL : teinte, saturation, valeur ; plus proche de la perception humaine.

Lab : perceptuellement uniforme.

CMYK : utilisé en impression,

YCbCr **:** utilisé pour la compression vidéo.

I1I2I3 : utile en analyse d'images.

Etc,

Après la lecture du contenu de ces quatre sites web :

- **https://www.makingsoftware.com/chapters/color-spaces-models-and-gamuts?utm_source=chatgpt.com**
- **https://patapom.com/blog/Colorimetry/ColorSpace/?utm_source=chatgpt.com**

- https://ciechanow.ski/color-spaces/?utm_source=chatgpt.com
- https://www.rapidtables.com/convert/color/index.html

  - Écrire une fonction **lister_espaces_couleurs() :** Pour chaque espace, inclure :

1. Composantes (ex. R, G, B ; H, S, V ; L, a, b…)
2. Intervalle de valeurs (ex. 0-255, 0-1, -128 à 127)
3. Avantages / usages typiques (vision humaine, compression, impression…)
4. Inconvénients / limites (non perceptuellement uniforme, sensibilité aux variations d'éclairage…)
5. **….**

  - Écrire une fonction **lister_conversion_espaces()** qui

1. liste des conversions possibles (ex. RGB → HSV, HSV → Lab, etc)
2. donne pour chaque conversion :
   1. La fonction OpenCV correspondante (cv2.COLOR_X2Y)
   2. Les limitations ou précautions (perte de précision, ordre des canaux, plage des valeurs)

  - Écrire une fonction **affichage_image_diff_espaces()** qui crée une figure montrant une image dans plusieurs espaces de couleur côte à côte : « Original RGB, HSV, Lab, YCbCr, niveaux de gris… »
  - Écrire une fonction **Lister_autres_espaces ()** qui affiche la liste des espaces couleurs pour lesquels, il n y a pas de fonction OpenCV.

c. Conversion entre espaces de couleurs :

i. Conversion Couleur → Niveau de gris :

a) Écrire une fonction **conversion_manuelle_rgb2gray(image, methode = "moyenne")** qui effectue la conversion de l'espace couleur RBG vers Niveaux de gris de trois façons :

- De base "moyenne" : I = (R + G + B) / 3
- Pondérations selon la recommandation ITU-R BT.601 "BT601" : I = 0.299R + 0.587G + 0.114B
- Pondérations selon la recommandation ITU-R BT.709 "BT709" : I = 0.2126R + 0.7152G + 0.0722B

Attention : les formules produisent des valeurs flottantes mais les images en niveaux de gris sont généralement codées en entiers de 0 à 255 (uint8).

b) Écrire une fonction **comparer_conversion_gris (image)** qui

- génère les 3 conversions manuelles (moyenne, BT601, BT709)
- affiche côte à côte :
  - L'image originale,
  - Les trois conversions manuelles,
  - La conversion via OpenCV: cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
- Calcule et affiche l'image différence pour chaque méthode (valeur absolue pixel par pixel)

- Calcule l'erreur moyenne sur toute l'image

ii. Conversion Espace couleur 1 → Espace couleur 2 :

a) Écrire des fonctions **conversion_manuelle_(SC1)2(SC2) (image)** qui effectue la conversion de l'espace couleur 1 vers l'espace de couleur 2.

b) Comparer vos résultats avec les fonctions OpenCv, en affichant les images de différence ainsi que l'erreur moyenne.

d. Visualisation et décomposition des canaux

Lorsque l'on décompose une image couleur, chaque canal contient une information spécifique : certains traduisent la luminance (Luma), c'est-à-dire les détails de luminosité, tandis que d'autres représentent la couleur (Chroma) ou la chrominance. En observant séparément ces canaux, on peut visualiser comment chaque composante influence l'image originale et comprendre la manière dont ils interagissent pour produire la perception globale. Cette exploration révèle également que l'œil humain est bien plus sensible aux variations de luminance qu'à celles de couleur.

i. Visualisation de tous les canaux d'une image

a) Écrire une fonction **afficher_canaux(image, espace="RGB")** qui :

- affiche l'image originale,
- sépare et affiche tous les canaux de l'image (en niveaux de gris) dans des sous-graphes,
- s'adapte automatiquement au nombre de canaux (3, 4 ou plus).

Tester la fonction sur une image RGB (Les canaux sont-ils équilibrés ? Quel canal contient le plus de détails ?)

b) Écrire une fonction **afficher_canaux_couleur (image, espace="RGB")** qui :

- affiche l'image originale,
- affiche chaque canal colorisé selon sa composante principale :
  - pour RGB → R en rouge pur, G en vert pur, B en bleu pur,
  - pour HSV/HSL → H colorisé selon la teinte, S et L en gris
  - pour YCbCr → Cb en bleu, Cr en rouge, Y en gris
- affiche également les canaux en niveaux de gris à côté pour comparaison.

Tester la fonction sur une image en différents espaces. Quelle différence perceptuelle constatez-vous entre l'affichage en niveaux de gris et l'affichage colorisé ? Quelle composante semble la plus perceptible pour l'œil humain ?

ii. Visualisation d'un canal spécifique

a) Écrire une fonction **afficher_canal (image, canal =0, espace="RGB")** qui :

- affiche uniquement le canal choisi (dans cet exemple canal = 0 correspond à rouge « R »),
- indique le nom du canal affiché (exemple R : la composante rouge, H : la teinte, etc.)

Tester cette fonction sur différents canaux et différents espaces (RGB, HSV, Lab). Quels canaux semblent les plus informatifs selon l'espace choisi ? Peut-on reconnaître l'image à partir d'un seul canal ?

iii. Comparaison entre espaces de couleur

a) Écrire une fonction **visualiser_espace_complet(image, espace_cible = "HSV")** qui :

- convertit l'image dans l'espace spécifié (HSV, Lab, YCbCr, …),
- affiche l'image originale (RGB), l'image convertie,
- puis les canaux séparés (RGB) et ceux de cet espace (H, S, V ou autres).

Tester par exemple les espaces, HSV, Lab, YCbCr. Quelles différences observez-vous entre les espaces ? Le canal de luminance est-il plus contrasté que les autres ?

iv. Explorer des images multi-canaux

Certaines images scientifiques ou médicales contiennent plus de trois canaux (par exemple en IRM, en scanners médicaux ou en imagerie satellitaire multispectrale). Ces canaux représentent des mesures variées (température, concentration chimique, propriétés biologiques, densités, longueurs d'onde invisibles comme l'infrarouge). Ainsi un canal n'est pas forcément une couleur dans le spectre visible, mais plutôt une dimension de données par pixel. Le concept de canal devient alors une notion générale. On peut visualiser ces canaux en pseudo-couleur pour mieux interpréter les données, mais la couleur affichée est artificielle, pas ce que l'œil humain verrait naturellement. C'est juste une interprétation pour faciliter la lecture humaine.

Exemples d'images multi-canaux :

- **IRM (Imagerie par Résonance Magnétique) :** chaque "canal" peut représenter une séquence différente (T1, T2, FLAIR…), capturant différents types de tissus ou propriétés biologiques. « https://fastmri.med.nyu.edu/»
- **Scanners CT :** les canaux correspondent à des densités ou énergies différentes selon la coupe ou le protocole d'acquisition « https://www.nlm.nih.gov/research/visible/getting_data.html ».

- **Satellites multispectraux :** prennent des photos dans différentes bandes du spectre électromagnétique (infrarouge, proche infrarouge, etc.), dépassant le visible. Le canal infrarouge permet de détecter la végétation, mais l'œil humain ne voit pas l'infrarouge. On peut afficher ce canal en rouge pour visualiser les zones vertes de végétation « https://collections.sentinel-hub.com/ ».

a) Écrire une fonction **afficher_multicanaux_gris (image)** qui affiche séparément tous les canaux d'une image en niveaux de gris, de manière automatique, quel que soit le nombre total de canaux.
b) Écrire une fonction **afficher_canal_colorisé (image , canal = 0)** qui visualise le canal choisi sous différentes colorisations (en niveau de gris, en rouge, en vert et en bleu) pour mieux percevoir ses structures.
c) Écrire une fonction **afficher_reconstruction_RGB (image, canaux = (0,1,2))** qui recompose et affiche une image RGB à partir de trois canaux choisis manuellement. Les valeurs de chaque canal sont normalisées dans l'intervalle [0, 255] pour un rendu correct.

e. Quantification des espaces de couleurs.

La quantification des couleurs consiste à réduire le nombre de valeurs possibles pour les pixels d'une image, afin de simplifier son traitement, sa représentation et sa comparaison avec d'autres images.

Avant cette étape, une image possède une dynamique, c'est-à-dire l'étendue des valeurs possibles que peuvent prendre ses pixels.

- Pour une image en niveaux de gris codée sur 8 bits, la dynamique est de 256 valeurs (de 0 à 255).
- Pour une image couleur RGB, chaque canal (R, G, B) possède 256 niveaux, ce qui donne potentiellement $256^3 \approx 16{,}7$ millions de couleurs distinctes.

Cette dynamique définit donc la richesse chromatique et le degré de détail de l'image.

La quantification revient à réduire cette dynamique : on regroupe des valeurs proches en un nombre limité de niveaux ou de classes (ou « centroïdes ») et on remplace chaque couleur d'origine par la couleur moyenne de son groupe.

Cette opération permet de :

- réduire la taille mémoire et le nombre de couleurs distinctes,
- faciliter la création d'histogrammes de couleurs,
- préparer les images à des étapes d'analyse ou de recherche par le contenu.

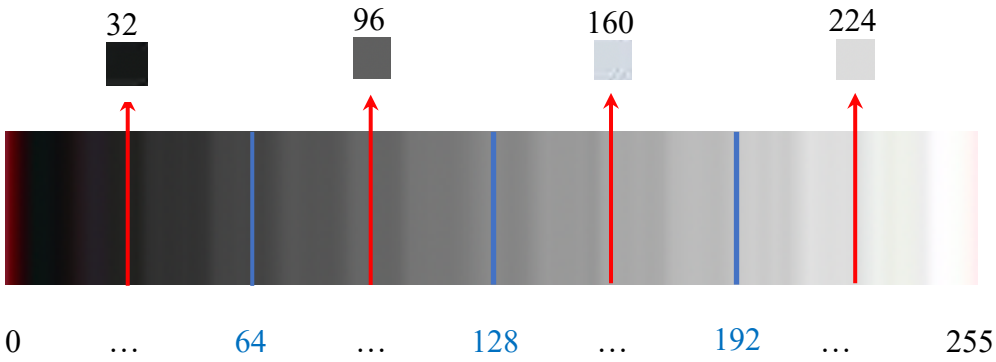
Les méthodes courantes de quantifications sont la quantification uniforme, la quantification vectorielle et la méthode adaptative. Dans la suite de ce projet, nous nous concentrons sur la quantification uniforme, c'est-à-dire une réduction régulière et équidistante des niveaux possibles dans l'espace de couleur.

I. Quantification d'une image en niveaux de gris

Cette opération consiste à diviser l'intervalle des intensités [0, 255] en un nombre fini de niveaux discrets. Chaque valeur de pixel est remplacée par la valeur moyenne de son intervalle.

Exemple :

Si l'on choisit 4 niveaux, les valeurs possibles seront : 32, 96, 160, 224. La quantification ramène toutes les valeurs proches à l'un de ces quatre niveaux.



250	127	22	3
50	193	120	35
60	127	110	70
150	130	165	200

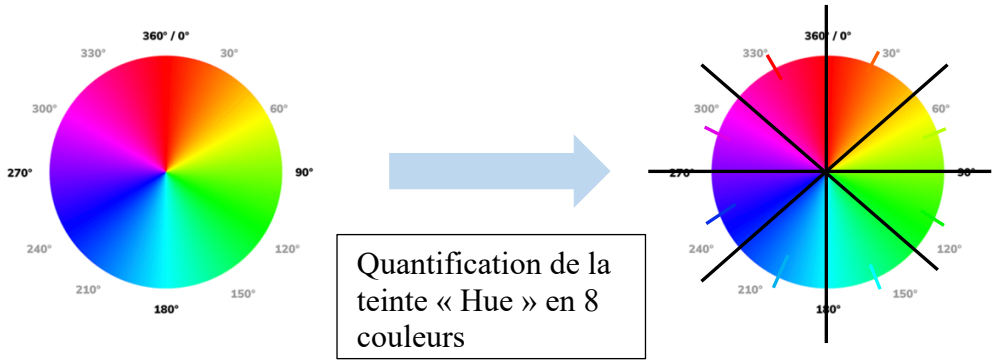
224	96	32	32
32	224	96	32
32	96	96	96
160	160	160	224

- Écrire une fonction **quantifier_gris (image, niveaux = 8)** qui prend une image en niveaux de gris (type uint8) et qui la quantifie en plusieurs intervalles, puis affiche ainsi l'image quantifiée ainsi que l'image originale.
- Tester cette fonction avec différents niveaux de quantification et observer la perte de détails.

II. Quantification d'une seule composante

Certaines composantes, comme la Teinte (Hue) dans l'espace HSV, peuvent être quantifiées séparément afin de réduire la diversité des couleurs dominantes tout en conservant la structure générale de l'image.

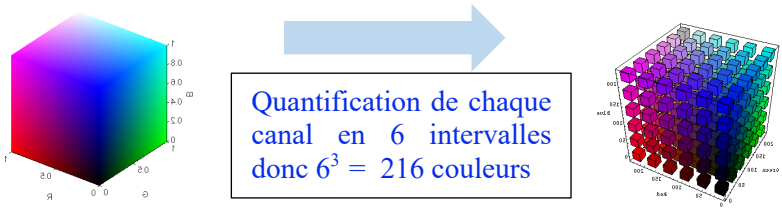
Exemple :



III. Écrire une fonction **quantifier_canal(image, canal=0, niveaux=12, espace='HSV')** qui convertit l'image dans l'espace choisi (par exemple HSV), sélectionne la composante correspondant au canal (H, S ou V), applique la quantification uniforme, puis affiche côte à côte l'image originale, la composante avant quantification, la composante après quantification et l'image reconstruite.

IV. Quantification d'une image couleur RGB

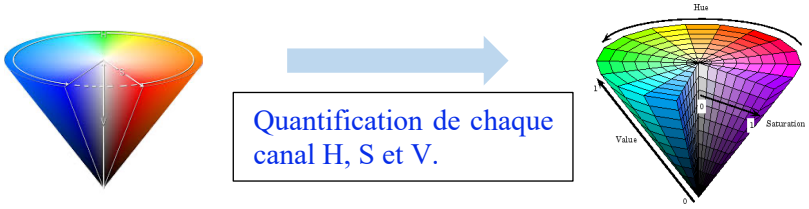
Pour une image couleur RGB, la quantification peut être appliquée indépendamment sur chaque canal (R, G, B). Le nombre de niveaux peut être identique pour les trois canaux, ou bien différent pour chacun d'eux, selon le degré de précision ou l'effet visuel souhaité.



- Écrire une fonction **quantifier_rgb (image, niveaux_R = 6 , niveaux_G = 6, niveaux_B = 6)** qui quantifie chaque canal selon le nombre de niveaux choisi et affiche le résultat. ester visuellement différentes combinaisons de paramètres, en utilisant par exemple le même nombre d'intervalles pour tous les canaux ou des valeurs différentes. Observer l'impact sur la fidélité des couleurs et la perte de nuances.

V. Quantification dans les espaces HSV / HSL

Les espaces HSV et HSL sont souvent plus adaptés à la perception humaine que l'espace RGB. On peut quantifier séparément les trois composantes :

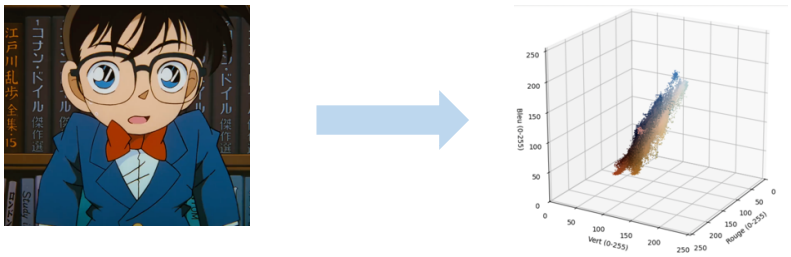


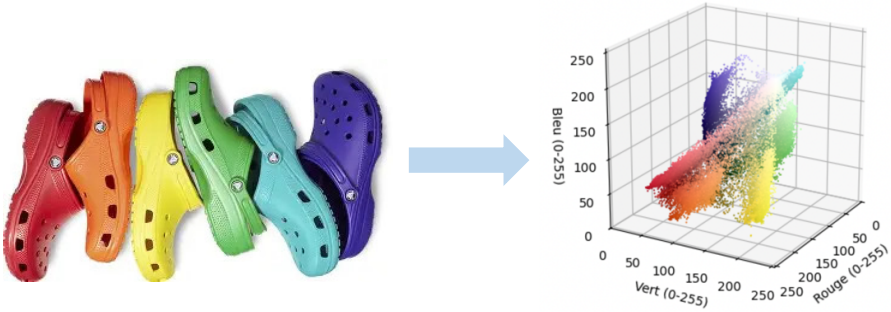
- Écrire une fonction **quantifier_hsv_hsl (image, niveaux_h=12, niveaux_s=4, niveaux_v=4, espace='HSV')** qui convertit l'image RGB vers HSV ou HSL, applique une quantification à chaque canal selon les paramètres choisis, reconvertit l'image dans l'espace RGB, puis affiche les résultats. Tester visuellement différentes quantifications et comparer les versions originale et quantifiée.

f. Visualisation 3D dans l'espace de couleur

La visualisation 3D d'une image dans un espace de couleur consiste à représenter chaque pixel comme un point dont les coordonnées correspondent à ses composantes colorimétriques (par exemple R, G, B dans l'espace RGB). Chaque axe du plan tridimensionnel correspond à un canal de couleur, et la couleur du point affiché correspond à la couleur réelle du pixel, même si les coordonnées proviennent d'un autre espace (HSV, Lab, YCbCr…). Cela permet de visualiser la forme et la densité du nuage de couleurs dans différents espaces, tout en conservant la perception des couleurs. Selon la taille de l'image, on peut afficher tous les pixels ou seulement un échantillon de ceux-ci pour alléger la visualisation.

Exemples :





i. Nuage de points complet

Tous les pixels de l'image sont utilisés pour former le nuage 3D. Chaque point correspond à un pixel, positionné selon ses coordonnées de couleur, et coloré selon sa valeur RGB d'origine. L'utilisation de tous les points permet une représentation fidèle et exhaustive de la distribution des couleurs. Cependant pour les images de grande taille, le nombre élevé de points (plusieurs millions) rend la visualisation lente et peu lisible.

- Écrire une fonction **nuage_3d_espace(image, espace='HSV')** qui affiche le nuage de points de tous les pixels dans l'espace de couleur choisi (RGB, HSV, Lab, YCbCr…), avec la couleur réelle des pixels dans l'image RGB d'origine.

ii. Nuage de points échantillonné

Pour les images volumineuses, il est souvent préférable d'afficher seulement un sous-ensemble de pixels, afin de réduire le temps de calcul et d'améliorer la lisibilité du nuage. Deux méthodes principales peuvent être utilisées : l'échantillonnage aléatoire et l'échantillonnage uniforme.

1. Échantillonnage aléatoire

Les pixels sont sélectionnés de façon aléatoire dans l'image selon un taux d'échantillonnage fixé (par exemple 10 %).
Cette méthode conserve la distribution globale des couleurs tout en réduisant la densité du nuage.

- Écrire une fonction **nuage_3d_espace_aleatoire(image, espace='HSV' , taux=0.1)** qui sélectionne aléatoirement un pourcentage de pixels et affiche le nuage de points dans l'espace de couleur choisi (RGB, HSV, Lab, YCbCr…), avec la couleur réelle des pixels.

2. Échantillonnage uniforme

L'échantillonnage uniforme consiste à prélever les pixels selon une grille régulière dans l'image (par exemple un pixel sur dix dans chaque direction). Cette méthode garantit une couverture homogène de l'image entière, indépendamment de la distribution des couleurs.

- Écrire une fonction **nuage_3d_espace_uniforme(image, espace='HSV' , pas=10)** qui sélectionne un pixel sur « **pas** » lignes et colonnes, puis affiche le nuage de points dans l'espace de couleur choisi (RGB, HSV, Lab, YCbCr…), avec la couleur réelle des pixels.

3. Visualisation interactive

Une visualisation 3D peut être rendue interactive afin de permettre la rotation, le zoom et l'exploration du nuage sous différents angles.
Cela facilite l'interprétation de la répartition des couleurs dans l'espace.

- Écrire une fonction **afficher_nuage_3D_interactif(image, espace="RGB")** qui crée un nuage de points 3D interactif avec rotation et zoom, en utilisant par exemple la bibliothèque `plotly`.

iii. Visualisation 3D d'une image quantifiée

Après la quantification des couleurs, le nuage de points 3D devient moins dense : les couleurs proches sont regroupées, et les zones du cube RGB ou du cône HSV apparaissent plus structurées.
Cette visualisation permet de comparer directement la répartition des couleurs avant et après quantification, d'observer l'impact de la réduction des niveaux sur la diversité colorimétrique, et d'interpréter visuellement les pertes d'information ou les regroupements de teintes.

- Écrire une fonction **nuage_3d_image_quantifiee(image, niveaux_R=6, niveaux_G=6, niveaux_B=6, espace='RGB')** qui applique une quantification de couleur puis affiche côte à côte le nuage original et celui de l'image quantifiée dans l'espace choisi.

g. Tableau de bord de visualisation et d'analyse colorimétrique

Après avoir étudié séparément la représentation des images, leur quantification et leur visualisation 3D, il est instructif de regrouper ces éléments dans un tableau de bord unique. Ce tableau permet de comparer plusieurs aspects d'une même image en un seul coup d'œil :

- l'image originale,
- la version quantifiée,
- leurs représentations dans différents espaces de couleurs,
- et leurs nuages de points dans l'espace colorimétrique.

Cette première partie sur la représentation et exploration numérique d'une image a permis d'explorer la structure interne des images, les espaces de couleur et leur visualisation. Ces connaissances sont fondamentales pour la prochaine séance : l'extraction de caractéristiques utilisées pour comparer les images selon leur contenu.

b. Exploration pratique des formats d'images

Après avoir aborder les espaces de couleurs (RGB, HSV, Lab…), intéressons-nous maintenant aux formats de stockage d'image (.jpg, .png, .bmp, etc.). Ces formats déterminent comment une image est enregistrée sur disque, mais ne décrivent pas la manière dont les couleurs sont représentées en mémoire pendant le traitement.

Chaque format possède ses caractéristiques propres et repose sur un mode de codage spécifique (compression, nombre de bits, présence d'un canal alpha, métadonnées, etc.) Le choix du format influence la taille sur disque, la qualité visuelle, et la compatibilité avec d'autres logiciels.

On distingue généralement deux grandes familles:

- **Formats sans perte (lossless)** : préservent intégralement les valeurs des pixels (PNG, BMP, TIFF).
- **Formats avec perte (lossy)** : réduisent les informations pour diminuer la taille (JPEG, WebP).

Comprendre ces distinctions est essentiel pour choisir le format le plus adapté selon le contexte : affichage, impression, archivage, diffusion ou traitement et analyse scientifique.

a. Exploration et comparaison de différents formats

Avant d'analyser les effets de la compression, commençons par comparer plusieurs formats de stockage afin d'observer leurs différences de taille.

- Écrire une fonction **comparer_formats(image_path)** qui charge une image originale (par exemple en format « .png ») et qui l'enregistre dans plusieurs formats : « .jpg », « .bmp », « .tiff», « .webp ».
- Comparer les tailles (en octets) des fichiers obtenus, qu'observez-vous ?

b. Effets de la compression

La compression permet de réduire la taille d'un fichier image, mais elle a un impact direct sur la qualité visuelle.

- **Le format JPEG**

Le format **JPEG** est l'un des plus utilisés pour les photos numériques et le web. Il repose sur une **compression avec perte** : certaines informations sont supprimées pour réduire la taille du fichier. Cette perte est souvent invisible à l'œil nu à faible compression, mais devient visible lorsque la compression est forte.

Deux phénomènes sont à comprendre ici :

1. **Le taux de compression initial** : plus il est élevé, plus la taille du fichier diminue, mais la qualité visuelle se dégrade (apparition d'artefacts, zones floues ou bruitées, perte de texture).
2. **La re-compression répétée** : à chaque réouverture et sauvegarde d'un fichier JPEG, même avec le même taux de qualité, l'image subit une nouvelle perte d'information. Cette dégradation cumulative et irréversible, finit par altérer significativement l'image, c'est ce qu'on appelle souvent "l'effet de génération".

- Écrire une fonction **compression_jpeg(image_path, niveaux_qualite=[95, 80, 60, 40, 20])** qui

    - sauvegarde une image en JPEG à différents niveaux de qualité,
    - compare les tailles des fichiers obtenus.
    - compare les images résultantes par rapport à l'image originale :
      - calcule et affiche l'image différence (valeur absolue pixel par pixel),
      - calcule l'erreur moyenne sur l'ensemble de l'image.
    - présente côte à côte les différentes images résultantes pour visualiser l'évolution de la qualité.
- Écrire une fonction recompression_recursive(image_path, n_iter=5, qualite=80) qui :
    - commence à partir d'une image initiale (par ex. image.png),
    - l'enregistre une première fois en JPEG (image_1.jpg) avec la qualité spécifiée,
    - puis, à chaque itération :
      - rouvre l'image compressée précédente (image_i.jpg),
      - la réenregistre sous un nouveau nom (image_{i+1}.jpg),
      - répète l'opération n_iter fois.
- Ensuite :
    - compare toutes les versions obtenues (image_1.jpg, image_2.jpg, …, image_n.jpg) à l'image originale .png :
      - calculer et afficher l'image différence et l'erreur moyenne,
      - observer visuellement la dégradation progressive.
    - présente côte à côte les différentes versions pour visualiser l'évolution de la qualité.
- Tester les deux fonctions sur différentes images (naturelles, icones, visages, etc).
- Identifier à partir de quel seuil ou après combien de réenregistrements la dégradation devient-elle visible et quelles zones sont le plus affectés (uniformes, contours, colorées, texturées) ?

- **Autres formats compressés**

De nouveaux formats comme WebP, HEIC ou AVIF ont été développés pour offrir une meilleure compression que JPEG : ils produisent des fichiers plus petits pour une qualité visuelle équivalente ou supérieure. Cependant, ces formats sont encore moins compatibles avec de nombreux logiciels, navigateurs ou systèmes, ce qui explique pourquoi le JPEG reste le format le plus utilisé pour la diffusion d'images.

- Écrire une fonction comparer_formats_compresses (image_path, qualite=80) qui :
    - charge une image originale au format .png,

    - l'enregistre dans plusieurs formats compressés : .jpg, .webp, .avif (si disponible),
    - mesure et affiche la taille (en octets) de chaque fichier obtenu.
    - affiche ensuite les images côte à côte pour comparer la qualité visuelle, la fidélité des couleurs, et noter les différences de rendu selon les formats.
- Écrire une fonction recompression_formats(image_path, n_iter=5, qualite=80) qui :
    - part de l'image originale .png,
    - pour chaque format choisi (.jpg, .webp, .avif) :
        - crée une première version compressée (image_format_1.ext),
        - puis répète n_iter fois le processus :
            - ouvrir l'image compressée précédente,
            - la sauvegarder à nouveau dans le même format et au même taux de qualité,
            - enregistrer chaque version avec un indice (image_format_2.ext, image_format_3.ext, etc.).
    - Comparer ensuite :
        - les tailles de fichiers à chaque itération,
        - les différences visuelles entre l'image originale et les versions successives,
        - la dégradation cumulative pour chaque format.

Quel format résiste le mieux à la re-compression répétée ? À partir de combien d'itérations les dégradations deviennent-elles visibles ? Quels compromis observez-vous entre qualité, poids du fichier, et compatibilité logicielle ?

c. Analyse des métadonnées

Certains formats (notamment JPEG et TIFF) peuvent contenir des métadonnées EXIF : informations sur l'appareil photo, la date, les réglages, voire la géolocalisation.

1. Écrire une fonction **lire_metadonnees(image_path)** qui affiche les principales métadonnées EXIF d'une image.
2. Quelles informations supplémentaires sont présentes dans le fichier ?
3. Pourquoi certaines images (issues du web ou modifiées) n'ont-elles plus de métadonnées ?